

Pestaña 1

enviar pdf (para el lunes 30)

- presentación y definir un nombre de grupo
- redactar mínimo una plana, la problemática
- redactar y explicar un listado de funcionalidades de la potencial solución
- redactar y explicar un listado de potenciales usuarios de la potencial solución

3. Mal estado de las calles en Arica

Nombre Grupo: Atlas Arica

Zona crítica: Ciudad de arica

Problema:KL

- Mal estado de las calles en la ciudad de arica
- Pocas soluciones y mala gestión en reparar las calles en mal estado

Soluciones: Una aplicación que permita hacer seguimiento de control de pavimentación, identificando zonas críticas para poder definir con mayor certeza las áreas prioritarias de intervención, para poder administrar de manera más eficiente los fondos sectoriales ya sea municipales, gore y serviu y además generar un seguimiento georeferenciado con zonas de alerta donde se puedan advertir o programar intervenciones de mayor o de menor grado.

Nombre del Grupo:

ATLAS ARICA

Problemática

Mal estado de las calles de la ciudad de Arica

Integrantes:

Victor Breems

Willy Cruz

Gabriel Delgado

Nicolas Olivares

Problemática: Mal estado de las Calles de la Ciudad de Arica

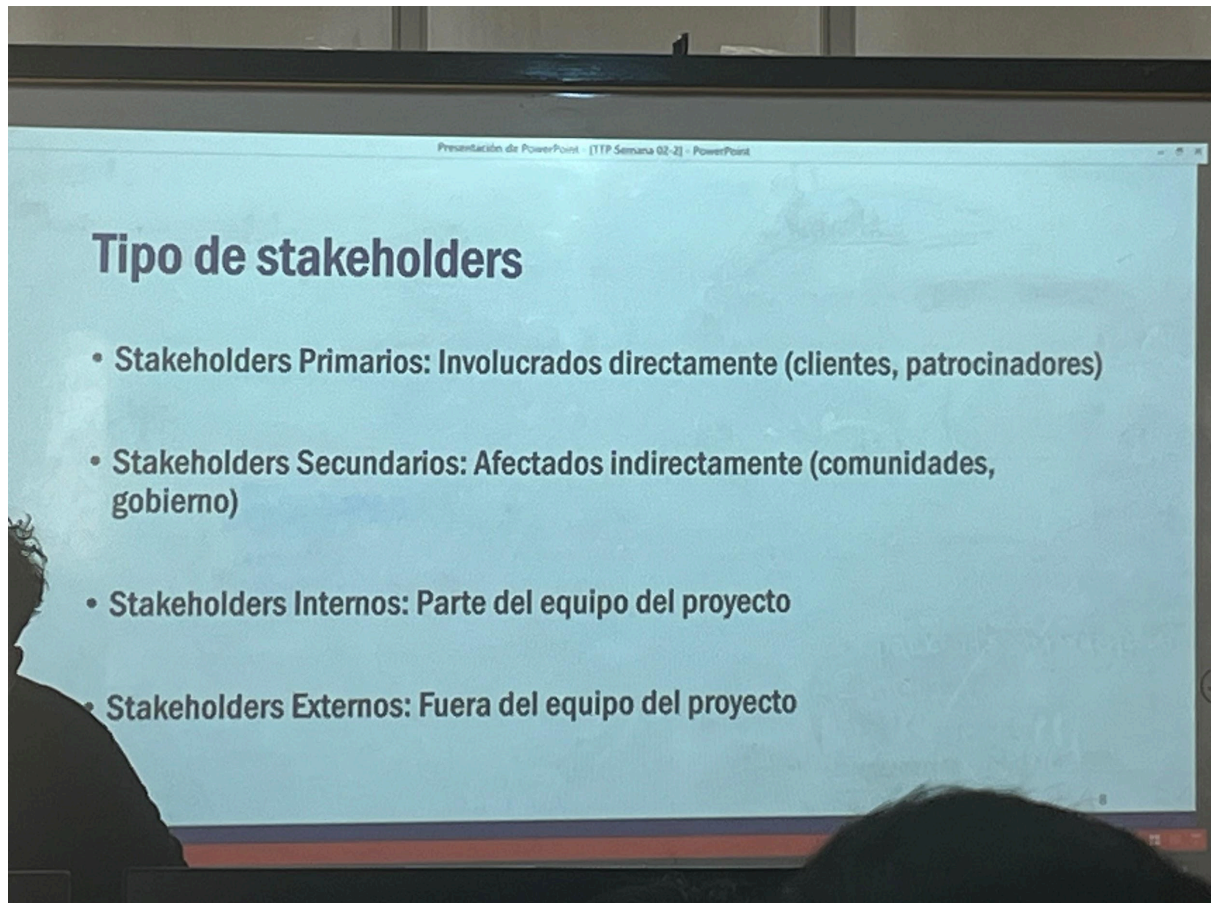
En la ciudad de Arica, el estado de las calles presenta un deterioro constante, lo que genera múltiples problemas tanto para conductores como peatones. Baches, grietas y pavimentos en mal estado afectan la seguridad vial, aumentan el desgaste de los vehículos y disminuyen la calidad de vida de los habitantes.

Uno de los principales problemas es la falta de una gestión eficiente en la mantención y reparación de las calles. Actualmente, las intervenciones suelen realizarse de forma reactiva, es decir, cuando el daño ya es evidente o grave, lo que implica mayores costos y menor eficiencia en el uso de los recursos.

Además, existe poca información sistematizada sobre el estado real de los pavimentos en la ciudad. No se cuenta con un seguimiento histórico que permita identificar qué zonas se deterioran más rápido, cuáles han sido intervenidas anteriormente o cuáles requieren atención urgente.

Otro factor importante es la falta de priorización clara al momento de intervenir. Muchas veces no se tiene certeza de qué sectores son realmente críticos, lo que dificulta una correcta distribución de los recursos provenientes de distintas entidades como municipalidades, el Gobierno Regional (GORE) y el SERVIU.

Finalmente, la ausencia de herramientas tecnológicas con georreferenciación impide visualizar de manera clara las zonas



problemáticas, dificultando la planificación estratégica y la prevención de futuros daños.

Por esta razón, se propone el desarrollo de una solución tecnológica que permita mejorar la gestión del estado de los pavimentos en la ciudad.

Solución: ATLAS Arica

ATLAS Arica es una aplicación móvil que permitirá realizar un seguimiento del estado de las calles en la ciudad mediante el uso de información georreferenciada y registros históricos.

Esta aplicación tendrá como objetivo principal identificar zonas críticas, facilitar la toma de decisiones y optimizar la gestión de los recursos destinados a la mantención de pavimentos.

Funcionalidades:

1. Mapa georreferenciado

Visualización de las calles en un mapa con su estado actual.

2. Seguimiento histórico

Registro del estado de las calles a lo largo del tiempo.

3. Identificación de zonas críticas

Detección de sectores con mayor deterioro.

4. Sistema de alertas

Avisos cuando una zona comienza a empeorar.

5. Priorización de intervenciones

Ordena las calles según urgencia.

6. Apoyo en la gestión de recursos

Mejor uso de fondos municipales, GORE y SERVIU.

Usuarios potenciales

- 1) Municipalidades
- 2) SERVIU
- 3) Gobierno Regional (GORE)
- 4) Empresas constructoras
- 5) Ciudadanos (beneficio indirecto)

Tipo de Stakeholders – ATLAS ARICA

Stakeholders Primarios (Involucrados directamente):

- Municipalidades
- SERVIU
- Gobierno Regional (GORE)
- Empresas constructoras

Stakeholders Secundarios (Afectados indirectamente):

- Ciudadanos de la ciudad de Arica
- Conductores y peatones

- Comercios locales

Stakeholders Internos (Parte del equipo del proyecto):

- Victor Breems
- Willy Cruz
- Gabriel Delgado
- Nicolas Olivares

Stakeholders Externos (Fuera del equipo del proyecto):

- Proveedores de tecnología (desarrollo de software, servidores, etc.)
- Organismos públicos asociados
- Empresas de mantenimiento vial

Resumen completo del IEEE Std 830-1998

(Recomendaciones para Especificaciones de Requisitos de Software)

1. Propósito y alcance

Este estándar describe qué debe contener una buena especificación de requisitos de software (SRS) y cómo debe ser. Es aplicable tanto a software desarrollado a medida como a la selección de productos comerciales o internos. No prescribe métodos, nomenclaturas ni herramientas concretas.

2. Definiciones clave

- Contrato: documento legal entre cliente y suministrador.
- Cliente: quien paga el producto (puede ser interno).
- Suministrador: quien produce el producto.
- Usuario: quien opera o interactúa directamente con el software.

3. Consideraciones para producir una buena SRS

3.1 Naturaleza de la SRS

Debe responder a:

- Funcionalidad: qué hace el software.
- Interfaces externas: cómo interactúa con personas, hardware, otros software.
- Rendimiento: velocidad, disponibilidad, tiempos de respuesta.
- Atributos: portabilidad, corrección, mantenibilidad, seguridad.
- Restricciones de diseño: estándares, lenguajes, políticas de BD, límites de recursos.

3.2 Entorno de la SRS

La SRS debe definir correctamente todos los requisitos, sin describir diseño ni implementación, y sin imponer restricciones adicionales (esas van en otros documentos como el plan de calidad).

3.3 Características de una buena SRS

Una SRS debe ser:

Característica	Significado
Correcta	Cada requisito es realmente exigido.
Inequívoca	Cada requisito tiene una única interpretación. Se recomienda glosario, lenguajes de especificación o herramientas (orientadas a objetos, procesos, comportamiento).
Completa	Incluye todos los requisitos significativos, respuestas a todas las entradas válidas/inválidas, y definiciones. Los "TBD" (por determinar) deben ir acompañados de condiciones, responsable y fecha de eliminación.
Consistente	Sin conflictos internos (ej. formatos contradictorios, acciones lógicamente opuestas, términos diferentes para el mismo concepto).
Clasificada por importancia/estabilidad	Cada requisito se identifica como esencial, condicional u opcional. También se puede indicar su estabilidad (frecuencia esperada de cambios).
Verificable	Existe un proceso finito y económico para comprobar que se cumple (evitar términos como "funciona bien", "normalmente").
Modificable	Estructura coherente, sin redundancias (o con referencias cruzadas explícitas), cada requisito separado.
Trazable	Hacia atrás (origen del requisito) y hacia adelante (identificador único para futuros documentos).

3.4 Elaboración conjunta

Cliente y suministrador deben trabajar juntos porque ninguno por separado suele tener todo el conocimiento necesario.

3.5 Evolución y prototipado

- La SRS puede evolucionar; deben especificarse cambios mediante un proceso formal.
- Los prototipos ayudan a obtener retroalimentación rápida y a reducir cambios posteriores.

3.6 No incluir diseño ni requisitos del proyecto

- Diseño (partición en módulos, flujo de datos, estructuras de datos) no debe ir en la SRS, salvo restricciones necesarias por seguridad o rendimiento.
- Requisitos del proyecto (costes, plazos, métodos, QA, criterios de aceptación) van en otros documentos (plan de proyecto, plan de calidad, etc.).

4. Partes de una SRS (estructura recomendada)

4.1 Introducción (Sección 1)

- Propósito: para qué sirve la SRS y audiencia.
- Alcance: nombre del producto, qué hace y qué no, beneficios, consistencia con especificaciones de nivel superior.
- Definiciones, siglas y abreviaturas.
- Referencias (documentos citados).
- Visión general de la organización del documento.

4.2 Descripción general (Sección 2)

- Perspectiva del producto: si es independiente o parte de un sistema mayor. Diagrama de bloques. Incluye interfaz: sistema, usuario, hardware, software, comunicaciones, memoria, operaciones (modos, respaldo), adaptación al sitio.
- Funciones del producto: resumen de las funciones principales (ej. mantenimiento de cuentas, facturación).
- Características de los usuarios: nivel educativo, experiencia, conocimientos técnicos.
- Restricciones: políticas reguladoras, limitaciones de hardware, interfaces, operación en paralelo, funciones de auditoría, control, requisitos de fiabilidad, seguridad, etc.
- Suposiciones y dependencias: factores externos que afectan los requisitos (ej. disponibilidad de cierto sistema operativo).
- Asignación de requisitos: qué requisitos podrían aplazarse a versiones futuras.

4.3 Requisitos específicos (Sección 3)

Es la parte más importante. Debe incluir:

- Interfaces externas (usuario, hardware, software, comunicaciones).
- Funciones (descripción detallada de cada función).
- Requisitos de rendimiento (medibles, ej. “el 95% de las transacciones en menos de 1 s”).
- Requisitos de base de datos lógica (tipos de información, frecuencia, integridad, retención).
- Restricciones de diseño (cumplimiento de estándares, formatos de informes, trazabilidad de auditoría).
- Atributos del sistema software: fiabilidad, disponibilidad, seguridad, mantenibilidad, portabilidad.

4.4 Organización de los requisitos específicos (Sección 3)

Se puede organizar de varias formas según el sistema:

- Por modo (entrenamiento, normal, emergencia).
- Por clase de usuario (pasajeros, mantenimiento, bomberos).
- Por objetos (pacientes, sensores, médicos – con atributos y funciones).
- Por características (llamada local, desvío, conferencia – con secuencias estímulo/respuesta).
- Por estímulo (pérdida de potencia, cizalladura del viento...).
- Por respuesta (todas las funciones que generan una salida, ej. nóminas).
- Por jerarquía funcional (diagramas de flujo de datos y diccionarios).

4.5 Información de apoyo

- Tabla de contenidos, índice, apéndices (muestras de I/O, estudios de coste, problemas a resolver, instrucciones de empaquetado). Debe indicarse si los apéndices son parte de los requisitos.

5. Anexos útiles

Anexo A: Plantillas de SRS

Proporciona estructuras alternativas para la Sección 3 según el enfoque elegido (modo, clase de usuario, objeto, característica, estímulo, respuesta, jerarquía funcional o combinaciones múltiples).

Anexo B: Guía de cumplimiento con IEEE/EIA 12207.1-1997

Explica cómo una SRS que cumple IEEE 830 también puede cumplir los requisitos de un “Software Requirements Description” (SRD) según 12207.1. Incluye tablas de correspondencia de contenidos y requisitos adicionales (fecha de emisión, historial de cambios, requisitos de cualificación, características físicas/ambientales, requisitos de empaquetado, etc.).

6. Lo que me sirve para hacer otro informe sobre IEEE 830

Para elaborar un informe propio basado en este estándar, los elementos más útiles son:

1. La lista de características de una buena SRS (correcta, inequívoca, completa, consistente, clasificada, verificable, modificable, trazable) – se puede usar como criterios de evaluación o checklist.
 2. La estructura recomendada (Introducción, Descripción general, Requisitos específicos, Apéndices) – sirve como plantilla base para cualquier SRS.
 3. Las formas de organizar los requisitos específicos (por modo, usuario, objeto, característica, estímulo, respuesta, jerarquía funcional) – muy útil para elegir el enfoque adecuado según el tipo de sistema.
 4. Los ejemplos de requisitos verificables (frente a los no verificables) – para redactar requisitos medibles.
 5. Las restricciones sobre qué no incluir (diseño, detalles de implementación, requisitos del proyecto) – evita errores comunes.
 6. El tratamiento de los TBD – cómo documentar información pendiente de forma controlada.
 7. Las tablas de correspondencia con 12207 – si se necesita alinear con otros estándares de ciclo de vida.
 8. La recomendación de elaboración conjunta – justificación para procesos colaborativos
-

Propuesta

Propuesta de Colaboración Tecnológica: Proyecto ATLAS Arica

Para: Bitumix S.A.

De: Equipo Atlas Arica (Victor Breems, Willy Cruz, Gabriel Delgado, Nicolas Olivares)

Asunto: Implementación de solución tecnológica para la gestión y seguimiento de pavimentos asfálticos en la ciudad de Arica.

1. Introducción y Contexto

La ciudad de Arica enfrenta un deterioro constante en su infraestructura vial, caracterizado por eventos y pavimentos en mal estado que afectan la seguridad y la calidad de vida. Actualmente, la gestión de reparación es predominantemente reactiva, interviniendo solo cuando el daño es grave, lo que eleva los costos operativos y disminuye la eficiencia de los recursos.

Ante la falta de información sistematizada y un seguimiento histórico de los pavimentos, el grupo **Atlas Arica** propone una alianza estratégica con **Bitumix S.A.** para implementar una herramienta de vanguardia en la planificación de obras viales.

2. La Solución: ATLAS Arica

ATLAS Arica es una aplicación móvil diseñada para el seguimiento georreferenciado del estado de las calles. Para una empresa constructora como Bitumix S.A., esta herramienta representa una ventaja competitiva al permitir:

- **Identificación de zonas críticas:** Detección precisa de sectores con mayor deterioro para priorizar contratos de mantenimiento.
- **Optimización de recursos:** Facilitar una administración eficiente de los fondos sectoriales provenientes de entidades como el SERVIU, GORE y Municipalidades.

3. Funcionalidades Detalladas

Nuestra plataforma ofrece un ecosistema de datos diseñado para la toma de decisiones estratégicas:

- **Mapa Georreferenciado:** Visualización actualizada del estado de las calzadas en toda la ciudad.
- **Seguimiento Histórico:** Registro de la evolución del pavimento a lo largo del tiempo, permitiendo predecir fallas futuras.
- **Sistema de Alertas Tempranas:** Notificaciones automáticas cuando una zona comienza a mostrar signos iniciales de deterioro, permitiendo intervenciones preventivas de menor costo.
- **Priorización de Intervenciones:** Algoritmo que ordena las calles según su urgencia.

4. Valor Agregado para Bitumix S.A.

Como potencial usuario clave, Bitumix S.A. podrá beneficiarse de:

1. **Planificación Estratégica:** Mejora en la programación de faenas de pavimentación de mayor o menor grado.
2. **Transparencia y Gestión:** Reportes detallados para justificar intervenciones ante los organismos mandantes (SERVIU y GORE).
3. **Liderazgo Tecnológico:** Posicionamiento como una constructora que utiliza inteligencia de datos para la conservación vial.

5. Conclusión

El proyecto ATLAS Arica no solo busca reparar calles, sino transformar la gestión vial de la región a través de la tecnología. Invitamos a Bitumix S.A. a ser parte de esta solución, optimizando sus procesos operativos y contribuyendo al desarrollo de una infraestructura urbana más resiliente en Arica.

STAKEHOLDERS

STAKEHOLDERS 🤨

Resumen

1. ¿Para qué sirve este documento?

El objetivo principal es que el **cliente** (quien paga por el programa) y el **proveedor** (quien lo fabrica) se pongan de acuerdo.

- **Evita malentendidos:** Asegura que ambas partes entiendan exactamente lo que el programa debe hacer.
- **Ahorra dinero y tiempo:** Es mucho más barato corregir un error en un papel que arreglar un programa que ya está terminado.
- **Sirve para revisar:** Cuando el programa esté listo, se usa este documento para verificar si el fabricante cumplió con todo lo prometido.

2. ¿Cómo debe ser una buena descripción del programa?

El texto dice que una buena descripción (SRS) debe tener estas características:

- **Correcta:** Todo lo que dice debe ser verdad y necesario para el programa.
- **Sin confusiones (No ambigua):** Cada frase debe tener una sola interpretación. No debe haber palabras que se presten a dudas.
- **Completa:** No debe faltar nada. Debe decir qué pasa si el usuario hace algo bien y también qué pasa si se equivoca.
- **Consistente:** No puede decir una cosa en la página 1 y lo contrario en la página 10.
- **Ordenada por importancia:** Debe quedar claro qué funciones son urgentes y cuáles pueden esperar.
- **Fácil de cambiar:** Debe estar organizada de forma que, si el cliente pide un pequeño cambio, sea fácil encontrar dónde anotarlo.

3. ¿Qué partes debe tener el documento final?

El estándar sugiere que el documento se divida en tres grandes secciones:

- **Sección 1: Introducción.** Explica para qué es el programa, a quién va dirigido y define palabras técnicas para que todos hablen el mismo idioma.
- **Sección 2: Descripción general.** Es una visión "desde lejos". Describe quiénes usarán el programa, qué limitaciones tiene (como el tipo de computadora) y qué cosas se dan por sentadas.
- **Sección 3: Requisitos específicos.** Esta es la parte más detallada. Aquí se explica:
 - **Interfaces:** Cómo se verá la pantalla y cómo se conectará con otros aparatos o programas.
 - **Funciones:** Exactamente qué cálculos o acciones hará.
 - **Rendimiento:** Qué tan rápido debe responder o cuánta memoria puede usar.

4. Consejos importantes para quienes lo escriben

- **Trabajo en equipo:** El cliente y el fabricante deben escribirlo juntos. El cliente sabe qué problema quiere resolver, pero el fabricante sabe qué es posible hacer técnicamente.
- **No mezclar con el diseño:** El documento debe decir **QUÉ** debe hacer el programa, pero no **CÓMO** programarlo por dentro (eso es tarea de los programadores más adelante).

- **No poner temas de dinero aquí:** El costo y las fechas de entrega van en otros documentos legales, no en la descripción técnica del programa.
- **Usar prototipos:** A veces es mejor hacer un dibujo o una versión "de juguete" del programa para que el cliente vea cómo quedará y pueda dar mejores ideas.

En resumen, el **IEEE 830** es la guía para que dos personas (cliente y creador) se entiendan perfectamente antes de gastar dinero y tiempo fabricando algo que quizás no era lo que se necesitaba.

1. Las Cualidades de un Requisito "Perfecto"

Para que un documento de requisitos sea útil, cada frase que escribas debe pasar por un filtro riguroso. La norma detalla características esenciales:

- **No ambigüedad:** Cada requisito debe tener una **única interpretación**. Si usas un término que puede significar dos cosas, debes incluirlo en un glosario con una definición específica. El lenguaje natural es traicionero, por lo que se recomienda que una persona independiente revise el texto para detectar frases confusas.
- **Verificabilidad:** Un requisito es verificable solo si existe un proceso finito y económico por el cual una persona o máquina pueda comprobar que el software lo cumple. Si dices "el sistema debe ser rápido", no es verificable. Si dices "el sistema debe responder en menos de 2 segundos", sí lo es.
- **Trazabilidad:** Debes poder rastrear cada requisito hacia atrás (¿de qué necesidad del cliente viene?) y hacia adelante (¿en qué parte del diseño y del código se implementó?). Esto se logra dándole a cada requisito un **identificador único** (como un número o nombre).
- **Priorización (Ranking):** No todos los requisitos valen lo mismo. Algunos son "esenciales" (si no están, el programa no sirve), otros son "deseables" (sería bueno tenerlos, pero no son críticos). Esto ayuda a los desarrolladores a decidir dónde poner más esfuerzo.

2. Estructura Profunda del Documento (SRS)

La norma propone organizar el documento en tres grandes bloques para que cualquier lector sepa dónde encontrar la información:

A. Introducción (Sección 1)

Aquí se define el "terreno de juego":

- **Alcance:** Identifica el producto por su nombre y explica qué hará y, muy importante, **qué no hará** el software.
- **Definiciones y Referencias:** Se listan todos los documentos técnicos usados como base y se aclaran términos técnicos para que el cliente no se pierda.

B. Descripción General (Sección 2)

Es una visión panorámica que no entra en detalles técnicos, sino en el contexto:

- **Perspectiva del Producto:** Si el software es parte de algo más grande (como el sistema operativo de un cajero automático), aquí se explica cómo encaja en ese conjunto.
- **Funciones del Producto:** Un resumen de alto nivel. Por ejemplo, si es un sistema contable, aquí dirías "permite gestionar facturas", pero sin explicar todavía cómo es la pantalla de facturación.
- **Restricciones:** Factores que limitan las opciones de los desarrolladores, como leyes gubernamentales, limitaciones de hardware (memoria escasa) o estándares de seguridad obligatorios.

C. Requisitos Específicos (Sección 3)

Esta es la sección más voluminosa y técnica. Debe contener detalles suficientes para que un diseñador empiece a trabajar y un tester sepa qué probar. Incluye:

- **Interfaces Externas:** Detalle minucioso de cada entrada y salida. Para cada dato, se debe definir su nombre, propósito, rango de valores válidos, precisión y formato de pantalla.
- **Requisitos de Rendimiento:** No solo velocidad, sino también cuántos usuarios puede soportar al mismo tiempo o cuántas transacciones por segundo debe procesar.
- **Atributos del Sistema:** Aspectos como la **Seguridad** (protección contra acceso no autorizado), la **Mantenibilidad** (qué tan fácil será arreglarlo en el futuro) y la **Portabilidad** (qué tan fácil es llevarlo a otra computadora).

3. Consideraciones Estratégicas

La norma IEEE 830 también da consejos sobre "cómo" trabajar:

- **Preparación Conjunta:** El documento debe ser un esfuerzo de equipo. Los clientes a menudo no saben qué es posible técnicamente, y los proveedores no conocen a fondo el negocio del cliente.
- **Uso de Prototipos:** Antes de cerrar el documento, es útil hacer prototipos (modelos rápidos). Esto genera retroalimentación inmediata, ya que los clientes suelen entender mejor un dibujo o una demo que un texto largo.
- **Evolución y Control de Cambios:** Un SRS no está "tallado en piedra". A medida que el proyecto avanza, surgirán cambios. La norma exige un **proceso formal de cambios** para que no se pierda el control de lo que se ha prometido.

- **Evitar el Diseño Prematuro:** El SRS debe decir **qué** debe hacer el sistema, no **cómo** debe estar programado por dentro. No debe decir qué estructuras de datos usar o cómo dividir el código en módulos, a menos que sea una restricción técnica absoluta del cliente.

4. Manejo de Incertidumbre (Los "TBD")

Es normal que al principio no se sepa todo. La norma permite usar la etiqueta **TBD (To Be Determined - Por determinar)**, pero advierte que un documento con TBDs no está completo. Para cada TBD, se debe anotar quién es responsable de resolverlo y en qué fecha debe estar listo.

Este enfoque estructurado asegura que, al final del día, el software construido sea exactamente lo que el cliente necesitaba, minimizando los costosos errores de "esto no es lo que yo pedí".

Tarea historias de usuario

$$\int_x^d$$

Historia de Usuario: Registro fotográfico de eventos viales

- **ID:** HU-07
- **Título:** Captura de fotos para reportes de baches
- **Narrativa:**
 - **Como** Operario de terreno / Administrador público
 - **Quiero** poder tomar y adjuntar fotografías de los eventos (baches o grietas) detectados en las calles
 - **Para que** exista una evidencia visual que permita validar la gravedad del deterioro y priorizar la intervención de forma más precisa.

Criterios de Aceptación:

- **C01:** El sistema debe permitir el acceso a la cámara del dispositivo móvil desde el formulario de reporte de eventos.
- **C02:** El usuario debe poder capturar una o más fotos por cada reporte generado.
- **C03:** Cada fotografía capturada debe quedar vinculada automáticamente a la ubicación georreferenciada del reporte.
- **C04:** El sistema debe permitir visualizar una miniatura de la imagen antes de enviar el reporte definitivo.
- **C05:** Las imágenes deben almacenarse en la base de datos vinculadas al historial del pavimento correspondiente.

Votación:

código		Victor	Willy Cruz	Nicolas Olivares	Gabriel Delgado	Decisión
C1	Registro fotográfico	3	8	13	1	4-5?
C2	Visualización Georreferenciada	8	13	5	20	

Historia de Usuario: Visualización Georreferenciada de Zonas Críticas

ID: HU-02

Título: Visualización de mapa con indicadores de estado

Narrativa:

- **Como** Administrador Público (SERVIU / Municipalidad)
- **Quiero** visualizar un mapa central de la ciudad con pines de colores verde, amarillo y rojo
- **Para** identificar de manera rápida y visual las calles con mayor deterioro y facilitar la toma de decisiones en la gestión vial.

Criterios de Aceptación:

- **C01:** El sistema debe integrar la API de **Google Maps SDK** para mostrar la cartografía actualizada de Arica.
- **C02:** El sistema debe renderizar pines de colores según el nivel de daño: **Verde** (Leve), **Amarillo** (Medio) y **Rojo** (Crítico).
- **C03:** El mapa georreferenciado debe cargar y mostrar hasta 100 puntos críticos en un tiempo máximo de 3 segundos bajo red 4G.
- **C04:** Al presionar un pin, el sistema debe desplegar la información básica del bache (calle y nivel de urgencia).
- **C05:** La visualización debe estar restringida exclusivamente a usuarios autenticados con sus credenciales.

04 | 05 | 26

historias de usuario video de telegram

como profesor quiero que el promedio general del alumno se muestre de color rojo si es inferior a 4.0 para identificar más fácilmente

IEEE 830 Propuesta

Especificación de Requisitos de Software (SRS)

Proyecto ATLAS Arica – Aplicación móvil para gestión y seguimiento de pavimentos asfálticos

Preparado para: Bitumix S.A.

Preparado por: Equipo Atlas Arica (Victor Breems, Willy Cruz, Gabriel Delgado, Nicolas Olivares)

Fecha: [Fecha actual]

1. Introducción

1.1 Propósito

El presente documento especifica los requisitos completos del software ATLAS Arica, una aplicación móvil destinada al seguimiento georreferenciado del estado de las calles en la ciudad de Arica.

El propósito es servir como base técnica para la colaboración con Bitumix S.A., permitiendo a la empresa planificar intervenciones viales de manera proactiva, optimizar recursos y posicionarse como referente en el uso de inteligencia de datos para la conservación vial.

La audiencia objetivo incluye:

- Área de planificación y operaciones de Bitumix S.A.
- Equipo de desarrollo de ATLAS Arica.
- Potenciales organismos mandantes (SERVIU, GORE, Municipalidades).

1.2 Alcance

El software ATLAS Arica permite:

- Visualizar en un mapa georreferenciado el estado actualizado de las calzadas.

- Registrar la evolución histórica del deterioro de los pavimentos.
- Generar alertas tempranas ante signos iniciales de deterioro.
- Priorizar calles según urgencia de intervención mediante un algoritmo.

El software no incluye:

- Gestión de contratos o facturación.
- Control de maquinaria o flotas.
- Recolección automática de datos sin intervención humana (los datos serán ingresados por inspectores o personal autorizado).

Aplicación: apoyo a la planificación de obras de mantenimiento y conservación asfáltica en la ciudad de Arica. Este documento es consistente con la propuesta de colaboración tecnológica presentada a Bitumix S.A.

1.3 Definiciones, siglas y abreviaturas

Término	Definición
ATLAS Arica	Nombre del proyecto y aplicación móvil.
Georreferenciación	Asignación de coordenadas geográficas a cada punto de la calzada.
Deterioro	Cambio en las condiciones del pavimento (grietas, baches, deformaciones).
Alerta temprana	Notificación automática cuando un sector muestra signos iniciales de deterioro.
Priorización	Ordenamiento de intervenciones según urgencia basado en algoritmo.
SERVIU	Servicio de Vivienda y Urbanización (ente mandante).

1.4 Referencias

- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications.
- Propuesta de Colaboración Tecnológica – Proyecto ATLAS Arica (documento interno del equipo).
- Normativa vial chilena (si aplica).

1.5 Visión general

El resto del documento se organiza así:

- Sección 2: Descripción general del producto, su contexto, funciones principales, usuarios y restricciones.
 - Sección 3: Requisitos específicos detallados (interfaces, funciones, rendimiento, atributos, etc.).
 - Apéndices (si se requieren).
-

2. Descripción general

2.1 Perspectiva del producto

ATLAS Arica es un sistema independiente pero que puede integrarse como módulo de información para sistemas de planificación de Bitumix S.A. o de los organismos públicos.

Diagrama de contexto conceptual:

text



2.1.1 Interfaces de sistema

- Entrada: Registro manual de estado de pavimento (con o sin fotos), coordenadas GPS.
- Salida: Mapas de calor, listados de calles priorizadas, alertas push.

2.1.2 Interfaces de usuario

- Pantalla principal: mapa interactivo con capas de colores según estado.
- Formulario de ingreso de incidencias (tipo de daño, severidad, ubicación, foto opcional).
- Panel de alertas (lista de notificaciones).
- Sección de reportes (exportables a PDF o Excel).

2.1.3 Interfaces de hardware

- GPS del dispositivo móvil (Android/iOS).
- Cámara para registro fotográfico (opcional).

2.1.4 Interfaces de software

- Sistema operativo móvil: Android 10+ o iOS 14+.
- Backend: servicio en la nube (AWS, Firebase u otro) para almacenamiento y procesamiento.
- API de mapas (Google Maps o OpenStreetMap).

2.1.5 Interfaces de comunicaciones

- Conexión a Internet (4G/5G/WiFi) para sincronización de datos y recepción de alertas.

2.1.6 Restricciones de memoria

- La app deberá funcionar con al menos 2 GB de RAM en el dispositivo.
- Almacenamiento local mínimo: 100 MB.

2.1.7 Operaciones

- Modo en línea (conectado) para sincronización y alertas en tiempo real.

- Modo fuera de línea opcional: permite registrar incidencias sin conexión y sincronizar después.

2.1.8 Requisitos de adaptación al sitio

- La aplicación debe permitir configurar límites de seguridad por zona (ej. umbrales de deterioro).
- Adaptable a distintos perfiles de usuario (inspector, planificador, gerente).

2.2 Funciones del producto

- Registro y georreferenciación de deterioros.
- Visualización en mapa con código de colores (verde = bueno, amarillo = deterioro inicial, rojo = crítico).
- Historial evolutivo por calle o sector.
- Generación de alertas tempranas (automáticas al superar umbral).
- Priorización de intervenciones según algoritmo (urgencia, tráfico, impacto).
- Reportes exportables para Bitumix y entidades mandantes.

2.3 Características de los usuarios

- Inspectores de campo: nivel educativo técnico o medio, con conocimientos básicos de uso de apps móviles y GPS.
- Planificadores / gerentes de Bitumix S.A.: profesionales con experiencia en gestión vial, requieren reportes claros y mapas interpretativos.
- Funcionarios de SERVIU / GORE: usuarios ocasionales que consultarán reportes y mapas, sin necesidad de ingresar datos.

2.4 Restricciones

- Políticas regulatorias: cumplimiento de normativa de protección de datos (Ley 19.628 en Chile).
- Plataforma objetivo: inicialmente Android, pudiendo extenderse a iOS.
- Idioma: español.
- Seguridad: autenticación de usuarios (correo + contraseña o cuenta institucional).
- Fiabilidad: la información debe ser consistente y respaldada diariamente.

2.5 Suposiciones y dependencias

- Se dispondrá de conectividad a Internet en la mayoría de las zonas de Arica.
- Los inspectores recibirán una capacitación básica para el uso de la app.
- Bitumix S.A. proporcionará retroalimentación para ajustar el algoritmo de priorización.
- Los datos históricos de pavimentos (si existen) pueden ser importados inicialmente.

2.6 Asignación de requisitos (versiones futuras)

- Versión 1.0: mapa, registro manual, alertas básicas, priorización simple.
 - Versión 2.0 (futura): integración con sensores IoT, machine learning predictivo, exportación automática a sistemas de SERVIU.
-

3. Requisitos específicos

3.1 Requisitos de interfaces externas

3.1.1 Interfaces de usuario

- Pantalla de inicio de sesión.
- Mapa táctil con zoom y selección de calles.
- Formulario de nuevo reporte con campos: tipo de daño (lista desplegable), severidad (leve/moderado/grave), coordenadas (automáticas o manuales), foto (opcional).
- Listado de alertas (con indicador visual de no leídas).
- Panel de priorización (lista ordenada de calles con puntaje).

3.1.2 Interfaces de hardware

- Uso de GPS con precisión de al menos 10 metros.
- Acceso a cámara (permiso opcional).

3.1.3 Interfaces de software

- Backend RESTful para sincronización.
- Servicio de notificaciones push (FCM para Android, APNS para iOS).

3.1.4 Interfaces de comunicaciones

- Protocolo HTTPS para todas las comunicaciones.

3.2 Requisitos funcionales

ID	Requisito
RF-01	El sistema permitirá registrar una incidencia con ubicación (GPS o manual), tipo de daño, severidad, fecha/hora y foto opcional.
RF-02	El sistema mostrará un mapa con capas de colores según el estado del pavimento (verde: bueno, amarillo: deterioro inicial, rojo: crítico).
RF-03	El sistema permitirá consultar el historial de evoluciones de una calle o sector (línea de tiempo).
RF-04	El sistema generará automáticamente una alerta temprana cuando se detecte un deterioro inicial (severidad leve) en una zona que previamente estaba en estado bueno.
RF-05	El sistema aplicará un algoritmo de priorización que considere: severidad, antigüedad del daño, cantidad de reportes en la misma zona y flujo vehicular estimado.
RF-06	El sistema permitirá exportar un listado priorizado de calles a formato PDF o CSV.

RF-07	El sistema mantendrá un registro histórico de todas las incidencias, con posibilidad de filtrar por fecha y sector.
RF-08	Los usuarios con perfil “inspector” solo podrán agregar reportes y ver mapas; los usuarios “planificador” podrán generar reportes y ver alertas; los usuarios “administrador” gestionarán usuarios y parámetros del algoritmo.

3.3 Requisitos de rendimiento

- Tiempo de respuesta: la carga del mapa (vista inicial) no superará los 3 segundos con conexión 4G.
- Sincronización: un reporte nuevo se sincronizará con el servidor en menos de 5 segundos en condiciones normales.
- Alerta temprana: se generará a más tardar 1 minuto después de que el servidor reciba un reporte que active la condición.
- Disponibilidad: el servicio backend tendrá una disponibilidad del 99% en horario laboral (08:00 a 20:00).
- Concurrencia: soportará al menos 20 usuarios simultáneos (crecimiento esperado).

3.4 Requisitos de base de datos lógica

- Entidades principales: Usuario, Reporte (incidencia), Calle/Sector, HistorialEstado, Alerta.
- Relaciones: un reporte pertenece a una calle/sector y a un usuario. Una alerta se asocia a un reporte.
- Integridad: no se permiten reportes sin ubicación válida.
- Retención: los datos se conservarán por un mínimo de 5 años.

3.5 Restricciones de diseño

- Estándares: se seguirán buenas prácticas de codificación para Android (Kotlin) y backend (Node.js/Python según decisión técnica).
- Trazabilidad de auditoría: cada cambio en el estado de una calle deberá quedar registrado con usuario, fecha y hora.

3.6 Atributos del sistema software

3.6.1 Fiabilidad

- El sistema no perderá datos de reportes ya sincronizados. En caso de caída del servidor, la app guardará localmente y reintentará.

3.6.2 Disponibilidad

- Se implementarán puntos de respaldo (backup) diarios de la base de datos.

3.6.3 Seguridad

- Autenticación mediante JWT (token con expiración).
- Las comunicaciones serán cifradas (TLS 1.2+).
- Roles y permisos estrictos (ver RF-08).

3.6.4 Mantenibilidad

- El código será modular y documentado. Se utilizará control de versiones (Git).

3.6.5 Portabilidad

- La lógica de negocio estará separada de la capa de interfaz para facilitar una futura versión iOS.

3.7 Organización de los requisitos específicos

Los requisitos se organizan por funciones principales (RF-01 a RF-08) y se complementan con tablas de rendimiento y atributos. No se requiere organización por modos o clases de usuario adicionales en esta versión.

Apéndices

Apéndice A: Ejemplos de formatos de reporte

Reporte priorizado de calles (ejemplo textual):

Calle	Sector	Puntaje urgencia (1-10)	Estado principal
Av. Diego Portales	Centro	9.2	Crítico
Calle 21 de Mayo	Sur	7.5	Deterioro inicial
...	...	...	...

Apéndice B: Supuestos técnicos adicionales

- Se utilizará Firebase Authentication para la gestión de usuarios.
- Los mapas se basarán en Google Maps API (requiere clave de API).
- El algoritmo de priorización inicial será: $\text{Urgencia} = (\text{Severidad} * 0.5) + (\text{Antigüedad_en_días} * 0.2) + (\text{Nro_reportes} * 0.3)$.

Apéndice C: Consideraciones legales y de privacidad

El manejo de datos personales (correos electrónicos, nombres) se ajustará a la Ley 19.628. No se recopilarán datos de localización de usuarios fuera del ámbito de reportes de pavimento.

primer sprint

Sprint 1: Infraestructura y Visualización Base

Objetivo: Establecer el entorno de desarrollo y la visualización cartográfica inicial.

- **Configuración de Entorno:** Creación del repositorio privado en **GitHub** y configuración de **Android Studio / Flutter**.
- **Módulo de Autenticación:** Implementación de **Supabase Auth** para gestionar los perfiles de Administradores Públicos y Operarios.
- **Visualización de Mapa (RF-01):** Integración de **Google Maps SDK** para mostrar el mapa de Arica con pines de prueba en colores Verde, Amarillo y Rojo.
- **Diseño UI:** Creación de los prototipos en **Figma** para la interfaz móvil.

sprint 1:

1. Configuración del entorno: Creación del repositorio privado en **GitHub**
2. **Configuración del primer mapa funcional.**

actividad evaluada 2

1. Requisitos en formato Historias de Usuario (Backlog)

A continuación, se presentan las historias clave alineadas con los Requisitos Funcionales (RF) de su informe:

Historia de Usuario: Autenticación y Seguridad

ID: HU-01

Título: Control de acceso por perfiles

Narrativa:

- **Como** Usuario del sistema (Municipalidad o Constructoras)
- **Quiero** iniciar sesión con mis credenciales mediante **Supabase Auth**
- **Para** acceder a las funciones correspondientes a mi rol y proteger la integridad de los datos viales.

Criterios de Aceptación:

- **C01:** El sistema debe validar el correo y contraseña contra la base de datos de Supabase.
- **C02:** Debe distinguir entre perfiles de "Administrador" (Municipalidad/SERVIU) y "Operario".
- **C03:** Si las credenciales son incorrectas, debe mostrar un mensaje de error y denegar el acceso.

Historia de Usuario: Algoritmo de Priorización

ID: HU-04

Título: Cálculo automático de urgencia

Narrativa:

- **Como** Administrador del SERVIU o GORE
- **Quiero** que el sistema asigne un nivel de prioridad a cada reporte mediante un algoritmo lógico
- **Para** optimizar la inversión de recursos públicos enfocándose en los daños más críticos primero.

Criterios de Aceptación:

- **C01:** El algoritmo debe considerar la profundidad del daño y el tiempo sin intervención.
- **C02:** El resultado debe categorizar el bache en niveles (Verde, Amarillo, Rojo).

- **C03:** La prioridad debe actualizarse en tiempo real al recibir nuevos datos de terreno.

2. Estimación de Esfuerzo (Story Points)

Utilizando la escala de Fibonacci, el equipo (**Víctor, Willy, Gabriel y Nicolas**) estima el esfuerzo para las historias principales:

ID	Historia de Usuario	Estimación (SP)	Justificación Técnica
HU-01	Autenticación (Supabase)	3	Configuración estándar de API de terceros.
HU-02	Mapa Interactivo (Google Maps)	5	Requiere manejo de renderizado y tiempos de respuesta (< 3s).
HU-03	Registro con GPS y Fotos (R2)	8	Integración de hardware (cámara/GPS) y almacenamiento en Cloudflare.
HU-04	Algoritmo de Priorización	5	Desarrollo de lógica matemática y backend.

3. Planificación de Sprints (Semestre)

Para cumplir con el desarrollo de **Atlas Arica** en el tiempo académico:

Sprint 1: Base de Datos y Visualización (2-3 semanas)

- **Objetivo:** Establecer la infraestructura y el mapa base.
- **Historias:** HU-01 (Autenticación) y HU-02 (Visualización de Mapa).
- **Entregable:** App con Login funcional y mapa de Arica con pines de prueba.

Sprint 2: Captura de Datos y Multimedia (3-4 semanas)

- **Objetivo:** Habilitar el reporte desde terreno.
- **Historias:** HU-03 (Registro con fotos/GPS) y HU-07 (Registro fotográfico).
- **Entregable:** Módulo de reporte funcional que sube imágenes a **Cloudflare R2**.

Sprint 3: Inteligencia y Reportes Finales (3-4 semanas)

- **Objetivo:** Implementar la lógica de negocio y cerrar el proyecto.
- **Historias:** HU-04 (Algoritmo de Priorización) y HU-05 (Reporte Tabular).
- **Entregable:** Aplicación completa con ranking de calles y documentación técnica entregada.